

Project Documentation: Finite State Machine (FSM) Module

Digital Design Lab

Project Overview

This design implements a parameterized **3-State Finite State Machine (FSM)** in Verilog HDL[cite: 30, 32]. The module manages three primary states (**S0/STATE_0**, **S1/STATE_1**, and **S2/STATE_2**) based on input signals **a** and **b**[cite: 30, 32]. Output **y1** operates primarily as a Moore-type output driven by current state, while **y0** functions as a Mealy-type output dependent on inputs **a** and **b** during state **S0**[cite: 30, 32].

Specifications & Functionality

- **States:**
 - **S0 (2'b00):** Initial state; outputs $y1 = 1$ [cite: 30, 32]. Sets $y0 = 1$ when both inputs $a = 1$ and $b = 1$ [cite: 30, 32]. Transitions to **S0** if $a = 0$, to **S2** if $a = 1, b = 1$, and to **S1** if $a = 1, b = 0$ [cite: 30, 32].
 - **S1 (2'b01):** Maintains state if $a = 0$; transitions to **S0** if $a = 1$ [cite: 30, 32]. Outputs $y1 = 1, y0 = 0$ [cite: 30, 32].
 - **S2 (2'b10):** Unconditional transition back to **S0** on next clock edge[cite: 30, 32]. Outputs $y1 = 0, y0 = 0$ [cite: 30, 32].
- **Asynchronous Active-Low Reset:** Forces state register to state **S0** immediately when reset line goes low[cite: 30, 32].

Port Summary

Port Name	Direction	Type	Width	Description
clk / clock	Input	wire	1	Clock input signal[cite: 30, 32]
reset / rst_n	Input	wire	1	Active-low reset signal[cite: 30, 32]
a / in_a	Input	wire	1	Control input line A[cite: 30, 32]
b / in_b	Input	wire	1	Control input line B[cite: 30, 32]
y0 / out_y0	Output	reg	1	Control output line 0[cite: 30, 32]
y1 / out_y1	Output	reg	1	Control output line 1[cite: 30, 32]

Table 1: FSM Module Port Interfaces

Verilog Source Code

Primary FSM Design (fsm.v)

```
module fsm (
    input wire a,
    input wire b,
    input wire clk,
    input wire reset,
    output reg y0,
    output reg y1
);
    localparam [1:0] S0 = 2'b00,
                   S1 = 2'b01,
                   S2 = 2'b10;

    reg [1:0] present_state, next_state;

    // Asynchronous Active-Low Reset State Register
    always @(posedge clk, negedge reset) begin
        case(reset)
            1'b0 : present_state <= S0;
            1'b1 : present_state <= next_state;
        endcase
    end

    task state (input [1:0] nextstate);
        begin
            next_state = nextstate;
        end
    endtask

    // Next State Logic
    always @(*) begin
        case(present_state)
            S0: begin
                if(!a)
                    state(S0);
                else if(b)
                    state(S2);
                else
                    state(S1);
            end
            S1: begin
                if(!a)
                    state(S1);
                else
                    state(S0);
            end
            S2: begin
                state(S0);
            end
            default:
                state(S0);
        endcase
    end

    // Output y1 Logic
    always @(*) begin
        case(present_state)
            S0: y1 = 1'b1;
        endcase
    end
endmodule
```

```

        S1: y1 = 1'b1;
        S2: y1 = 1'b0;
        default: y1 = 1'b0;
    endcase
end

// Output y0 Logic
always @(*) begin
    case(present_state)
        S0: begin
            if(b && a)
                y0 = 1'b1;
            else
                y0 = 1'b0;
            end
        S1: y0 = 1'b0;
        S2: y0 = 1'b0;
        default: y0 = 1'b0;
    endcase
end

endmodule

```

Refactored FSM Design (fsm2.v)

```

module fsm_2 (
    input wire in_a,
    input wire in_b,
    input wire clock,
    input wire rst_n,
    output reg out_y0,
    output reg out_y1
);

    localparam STATE_0 = 2'b00,
               STATE_1 = 2'b01,
               STATE_2 = 2'b10;

    reg [1:0] current_state, future_state;

    // State Register
    always @(posedge clock or negedge rst_n)
        if (!rst_n)
            current_state <= STATE_0;
        else
            current_state <= future_state;

    // Combined Next State & Output Logic
    always @(*) begin
        future_state = STATE_0;
        out_y0 = 0;
        out_y1 = 0;

        case (current_state)
            STATE_0: begin
                out_y1 = 1;
            end
        endcase
    end
endmodule

```

```
        future_state = (!in_a) ? STATE_0 : (in_b ? STATE_2 :  
            STATE_1);  
        out_y0 = (in_a && in_b);  
    end  
  
    STATE_1: begin  
        out_y1 = 1;  
        future_state = (!in_a) ? STATE_1 : STATE_0;  
    end  
  
    STATE_2: begin  
        future_state = STATE_0;  
    end  
endcase  
end  
  
endmodule
```

Testbench Module (fsm_test.v)

```
'timescale 1ns / 1ps  
  
module fsm_test;  
    reg a;  
    reg b;  
    reg clk;  
    reg reset;  
    wire y0;  
    wire y1;  
  
    fsm dut(  
        .a(a),  
        .b(b),  
        .clk(clk),  
        .reset(reset),  
        .y0(y0),  
        .y1(y1)  
    );  
  
    // Clock Generation  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    // Stimulus Logic  
    initial begin  
        a = 0;  
        b = 0;  
        reset = 0;  
  
        #10;  
        reset = 1;  
  
        #20;  
        a = 0;
```

```

    b = 0;

    #20;
    a = 1;
    b = 0;

    #20;
    a = 0;
    b = 1;

    #20;
    a = 1;
    b = 1;

    #20;
    a = 0;
    b = 0;

    #20;
    $stop;
end

// Monitor Display
initial begin
    $monitor("Time=%0t | reset=%b | a=%b b=%b | present=%b next=%b | \
y0=%b y1=%b",
            $time, reset, a, b, dut.present_state, dut.next_state,
            y0, y1);
end

endmodule

```

Simulation & Verification

Execution Command

To run simulation using Cadence Xcelium or ModelSim[cite: 31]:

```
xrun fsm.v fsm_test.v
```

Expected Simulation Output

The system monitor output tracks state switches and output signals dynamically[cite: 31]:

```

Time=0 | reset=0 | a=0 b=0 | present=00 next=00 | y0=0 y1=1
Time=10 | reset=1 | a=0 b=0 | present=00 next=00 | y0=0 y1=1
Time=50 | reset=1 | a=1 b=0 | present=00 next=01 | y0=0 y1=1
Time=55 | reset=1 | a=1 b=0 | present=01 next=00 | y0=0 y1=1
Time=70 | reset=1 | a=0 b=1 | present=01 next=01 | y0=0 y1=1
Time=90 | reset=1 | a=1 b=1 | present=01 next=00 | y0=0 y1=1
Time=95 | reset=1 | a=1 b=1 | present=00 next=10 | y0=1 y1=1
Time=105 | reset=1 | a=1 b=1 | present=10 next=00 | y0=0 y1=0

```